

Framework Density Visualization Specification

Plot-D4 vertex-occupancy and edge-length measures

mdstats

Contents

1	Purpose and status	1
2	Scientific definitions	2
2.1	Frame registration	2
2.2	Vertex occupancy measure	2
2.3	Edge-length measure	3
2.4	Why the channels are separate	3
3	Edge-source policies	3
3.1	Projected edges	3
3.2	Atom-resolved paths	4
4	Numerical discretization	4
4.1	Periodic cloud-in-cell deposition	4
4.2	Periodic Gaussian smoothing	4
5	Probability-mass visualization	5
6	Public API	5
7	Input constraints	6
8	Output model	6
9	Resource policy	6
10	Progress reporting	7
11	Validation requirements	7
12	Deliberate limitations	7
13	References	8
14	Euclidean metric and density resolution	8

1 Purpose and status

This document is the normative specification for Plot-D4, introduced in `mdstats` 0.19.32a0. Plot-D4 constructs time- or ensemble-averaged density measures for a persistent framework topology in the same registered coordinate system used by Plot-D1/D2 trajectories and Plot-D3 atomic occupancy fields. Version 0.19.39a0 couples Gaussian width to grid resolution and applies the same periodic spread-aware refinement policy to the framework vertex and edge channels.

Two scientific channels are defined:

1. **framework vertex occupancy**, normalized to one unit per projected framework vertex; and
2. **framework edge-length density**, normalized to the total retained framework arc length.

The channels are intentionally separate because they have different physical dimensions. They may be rendered together, but they are not numerically added without an explicit user-supplied conversion scale.

The implemented pipeline is

$$\begin{aligned}
 \text{topology-compatible frame geometry} &\longrightarrow \begin{cases} \text{vertex point measure,} \\ \text{edge arc-length measure,} \end{cases} \\
 &\longrightarrow \text{periodic cloud-in-cell grid} \\
 &\longrightarrow \text{normalized Gaussian smoothing} \\
 &\longrightarrow \text{probability-mass meshes.}
 \end{aligned}$$

Motive

A mean framework graph shows only the average geometry. It does not expose the spatial distribution swept out by vibrating framework atoms or by fluctuating framework bonds. Plot-D4 produces a fuzzy structural envelope around the mean graph while retaining exact scientific normalization.

The vertex channel is useful for visualizing anisotropic or non-Gaussian framework-atom motion. The edge channel is useful for visualizing the dynamic support of the framework network itself. Atom-resolved paths can reveal linker motion, whereas projected edges expose the corresponding coarse framework net.

2 Scientific definitions

2.1 Frame registration

All frames use the Plot-D1 registration boundary. Let H_t be the instantaneous cell and H_d the display cell. The continuous framework coordinates are first placed in one topology-consistent periodic gauge.

In material coordinates,

$$\mathbf{x}_i^{\text{mat}}(t) = \mathbf{f}_i^{(g)}(t)H_d.$$

In laboratory coordinates,

$$\mathbf{x}_i^{\text{lab}}(t) = \mathbf{f}_i^{(g)}(t)H_t,$$

followed by conversion into display-cell fractional coordinates for periodic voxel deposition. Framework-registered coordinates subtract the translational framework drift before applying H_d .

Trajectory frames use continuous topology-compatible placement. Ensemble frames are independently placed in the canonical reference gauge. Plot-D4 does not create temporal continuity for an ensemble.

2.2 Vertex occupancy measure

For projected framework vertices V and normalized frame weights w_t ,

$$\mu_V(d\mathbf{x}) = \sum_t w_t \sum_{v \in V} \delta(\mathbf{x} - \mathbf{x}_v(t)) d\mathbf{x}.$$

Its total measure is

$$\int_{\Omega} \rho_V(\mathbf{x}) d^3x = |V|.$$

The smoothed volume density has units

$$[\rho_V] = \text{\AA}^{-3}.$$

The vertex set is always the projected framework vertex set. Selecting an atom-resolved display graph does not silently promote linker atoms into the vertex channel.

2.3 Edge-length measure

For an instantaneous lifted edge or path segment e with arc-length parameterization $\gamma_{e,t}(s)$,

$$\mu_E(d\mathbf{x}) = \sum_t w_t \sum_{e \in E} \int_0^{L_e(t)} \delta[\mathbf{x} - \gamma_{e,t}(s)] ds d\mathbf{x}.$$

Its normalization is

$$\int_{\Omega} \rho_E(\mathbf{x}) d^3x = \sum_t w_t \sum_{e \in E} L_e(t).$$

Therefore,

$$[\rho_E] = \text{\AA}^{-2}.$$

The measure is invariant under edge orientation because reversing the parameterization leaves the line integral unchanged.

2.4 Why the channels are separate

The vertex and edge fields cannot be added directly:

$$[\rho_V] = \text{\AA}^{-3}, \quad [\rho_E] = \text{\AA}^{-2}.$$

A combined field would require an explicit reference length ℓ_0 ,

$$\rho_F = \rho_V + \frac{1}{\ell_0} \rho_E,$$

which introduces an additional modeling convention. Plot-D4 does not define a default ℓ_0 .

3 Edge-source policies

3.1 Projected edges

`edge_source="projected"` treats each decorated projected framework edge as one straight lifted segment joining its two framework vertices with the canonical periodic image shift.

This mode answers:

Where does the coarse framework net lie across the frame collection?

Parallel decorated edges and periodic self-image edges retain their scientific multiplicity.

3.2 Atom-resolved paths

`edge_source="atomic_paths"` expands each retained framework edge into the canonical atom-resolved path recorded by `FrameworkTopology`. Each adjacent atom pair contributes one lifted segment.

This mode answers:

Where do the chemically retained T–O–T or more general linker paths lie?

The path is taken from authoritative framework-topology provenance. Plot-D4 never reconstructs linker bonds from coordinates.

4 Numerical discretization

4.1 Periodic cloud-in-cell deposition

Both measures are deposited onto a uniform fractional grid using trilinear cloud-in-cell assignment. By default, the grid shape is derived from the display-cell vector lengths using the same target interval as atomic density:

$$N_i = \max\left(4, \left\lceil \frac{\|\mathbf{a}_i\|}{0.20 \text{ \AA}} \right\rceil\right).$$

Thus every realized lattice-grid edge is at most 0.20 angstrom. An explicit `grid_shape` overrides this automatic policy. The method is adapted from the particle-mesh construction described by Hockney and Eastwood (1988). Each sample contributes to the eight neighboring periodic grid nodes, and the assignment weights sum exactly to the sample weight.

For vertices, the sample weight is w_t . For an edge quadrature sample on a segment of length L divided into n equal parts, the sample weight is

$$\Delta\ell = \frac{w_t L}{n}.$$

Edge quadrature

A straight segment of physical length L is divided into

$$n = \max\left(1, \left\lceil \frac{L}{h_E} \right\rceil\right)$$

subintervals, where h_E is `edge_sample_spacing`. The midpoint of each subinterval is deposited with weight $w_t L/n$.

The discrete edge weights therefore sum exactly to the segment length before voxel interpolation. Increasing the sampling resolution changes spatial aliasing but not the stored total measure.

4.2 Periodic Gaussian smoothing

The deposited mass grid is convolved with a normalized Gaussian in reciprocal space. For reciprocal vector $\mathbf{k}$ and bandwidth σ ,

$$\widehat{K}_\sigma(\mathbf{k}) = \exp\left(-\frac{1}{2}\sigma^2|\mathbf{k}|^2\right).$$

The reciprocal vectors are computed from the complete display-cell matrix, so the smoothing is Cartesian-isotropic for triclinic cells. The result is renormalized to remove floating-point drift while preserving the exact scientific total.

When `gaussian_bandwidth=None`, the framework-density default derives the bandwidth from the longest realized lattice-grid interval,

$$\sigma = r_g h_{\max}, \quad r_g = 2.$$

The vertex trajectories also provide a spread-aware resolution diagnostic. For framework vertex i , the code computes the periodic Cartesian standard deviation

$$s_i = \sqrt{\frac{\text{tr } C_i}{3}},$$

using minimum-image displacements from the periodic Frechet mean. The default reference is the 10th percentile $s_{0.10}$. If the nominal Gaussian exceeds $0.5s_{0.10}$, the shared vertex/edge grid is refined and the Gaussian is reduced while preserving $\sigma/h_{\max} = 2$.

The refinement is limited by the per-channel share of `max_density_voxels`. If the requested spread criterion cannot be reached, the finest admissible grid is used and a warning records the residual $\sigma/s_{0.10}$ ratio. Explicit `grid_shape` and explicit `gaussian_bandwidth` values are preserved rather than silently changed.

5 Probability-mass visualization

Plot-D4 reuses the Plot-D3 highest-density-region construction adapted from Hyndman (1996). For target fraction q , the shell threshold c_q is chosen so that

$$\int_{\rho(\mathbf{x}) \geq c_q} \rho(\mathbf{x}) d^3x \geq q \int \rho(\mathbf{x}) d^3x.$$

The default shells enclose 50, 80, and 95 percent of the selected measure. These are not percentages of the maximum voxel value.

The periodic grid is seam-closed by appending a wrapped terminal plane along each axis. Two browser rendering policies are available.

The framework-density default is `render_mode="mesh"`. It extracts each shell before serialization as an explicit triangular surface using the Lewiner marching-cubes implementation in scikit-image. The method follows Lorensen and Cline's marching-cubes construction and Lewiner et al.'s topological ambiguity resolution. Plotly `Mesh3d` then performs only interactive triangle rendering. The prepared scalar fields do not depend on either backend.

Framework clouds use restrained opacity so the mean framework remains readable. Vertex and edge channels have independent legend groups and may be hidden separately.

6 Public API

The new public records are

```
FrameworkDensityOptions
FrameworkDensityFields
FrameworkDensity3DRenderOptions
```

The scene preparation API is extended as follows:

```
scene = prepare_framework_dynamics_scene(
    collection,
    framework_topology,
    framework_density_options=FrameworkDensityOptions(
        grid_interval=0.20,
        gaussian_bandwidth=None,
        gaussian_to_grid_ratio=2.0,
        adaptive_smearing=True,
        max_smearing_to_sample_sd_ratio=0.50,
        sample_sd_quantile=0.10,
        include_vertex_density=True,
        include_edge_density=True,
        edge_source="atomic_paths",
```

```

        edge_sample_spacing=0.20,
    ),
)

```

The returned scene owns

```

scene.framework_density_fields.vertex_density
scene.framework_density_fields.edge_length_density

```

Rendering uses

```

result = plot_framework_dynamics_3d(
    scene,
    framework_density_options=FrameworkDensity3DRenderOptions(
        mass_fractions=(0.50, 0.80, 0.95),
    ),
)

```

Trace provenance is available through

```

result.framework_density_trace_indices

```

7 Input constraints

The first backend requires:

- a valid `AtomisticFrameCollection`;
- one persistent `FrameworkTopology` compatible with every selected frame;
- periodicity along all three axes, inherited from the current periodic voxel backend;
- a nonsingular display cell;
- positive normalized frame weights;
- stable projected graph identity and normalized periodic winding;
- stable atom-resolved path identity when `edge_source="atomic_paths"`;
- a positive edge sampling spacing;
- sufficient voxel, field, sample, and rendering resources.

Topology-changing trajectories must be partitioned before density preparation. Plot-D4 never averages incompatible framework graphs.

8 Output model

`FrameworkDensityFields` contains up to two immutable `PeriodicScalarField3D` records. Each record stores:

- stable field key and human-readable label;
- scalar values on the periodic fractional grid;
- display cell;
- total scientific measure;
- contributing atom identities;
- smoothing bandwidth;
- optional deposited sample positions;
- physical units;
- registration, source, quadrature, and normalization metadata.

The scientific field remains renderer-independent and can later be exported to NumPy, Gaussian cube, or VTK formats.

9 Resource policy

Preparation is transactional under the LD10 complete-scene runtime budget. Omitted memory, count, pair, block, cache, workspace, thread, and wall-time limits are derived from the current process/job allocation rather than

fixed to a benchmark structure. The default uses 80% of detected available memory, 80% of available CPUs, and a 1,200-second scene objective. Explicit low-level values may tighten but cannot expand the authoritative scene policy.

Before a large allocation, Plot-D4 checks together:

- all atomic, framework-vertex, and framework-edge density fields;
- source and edge-quadrature sample arrays;
- dense nodes or sparse blocks and packed values;
- planning, routing, stencil, cache, and kernel work;
- retained fields plus the largest transient construction workspace;
- contour workspaces, final-output reserve, and isolated worker-pool memory;
- conservative preparation and rendering wall-time estimates;
- separate post-replication browser face, vertex, trace, and HTML profiles.

The edge quadrature preflight is performed while segment lengths are evaluated. A failure raises `GraphComplexityError`; it does not silently increase sampling spacing, omit edges or shells, coarsen the scientific grid, or borrow capacity from a browser face budget. The normative cross-module policy is `density_runtime_resource_policy_ld10_spec.{md,pdf}`.

10 Progress reporting

`prepare_framework_density_fields(...)` accepts the shared `progress=` port and reports framework-vertex occupancy and framework-edge length as separate bounded work items. Events include the selected backend and Gaussian width after each channel completes. The port is inherited from framework-dynamics preparation when called as part of a composite scene.

The former string `progress_callback=` remains a deprecated compatibility alias. See `docs/specs/progress_spec.{md,pdf}` for the package-wide contract.

11 Validation requirements

The focused tests must verify:

1. vertex normalization,

$$\int \rho_V d^3x = |V|;$$

2. edge normalization,

$$\int \rho_E d^3x = \langle L_{\text{tot}} \rangle;$$

3. dimensional metadata for both fields;
4. projected versus atom-resolved path arc lengths;
5. integer wrapping invariance;
6. independent-ensemble support;
7. rigid framework-drift removal;
8. variable-cell material and laboratory behavior;
9. independent channel selection;
10. resource preflight;
11. explicit rejection of mixed periodicity;
12. independent Plotly trace groups and HTML serialization;
13. no regression in Plot-D1/D2, Plot-D3, framework, or graph rendering.

12 Deliberate limitations

The first backend does not implement:

- mixed periodic/nonperiodic convolution;
- nonuniform or adaptive grids;

- time-dependent frame weights;
- rolling-window framework densities;
- difference densities;
- a dimensionally mixed vertex-plus-edge scalar field;
- tube-radius or chemical-bond-order weighting;
- topology-state mixtures;
- direct VTK or cube export.

These limitations are explicit rather than approximated silently.

13 References

- Hockney, R. W., and Eastwood, J. W. *Computer Simulation Using Particles*. First edition, 1988. The cloud-in-cell particle-mesh assignment is adapted from this source.
- Hyndman, R. J. “Computing and Graphing Highest Density Regions.” *The American Statistician* **50** (1996), 120–126. DOI: 10.1080/00031305.1996.10474359.
- W. E. Lorensen and H. E. Cline. “Marching Cubes: A High Resolution 3D Surface Construction Algorithm.” *Computer Graphics* **21**, 163-169 (1987). DOI: 10.1145/37402.37422.
- T. Lewiner, H. Lopes, A. W. Vieira, and G. Tavares. “Efficient Implementation of Marching Cubes’ Cases with Topological Guarantees.” *Journal of Graphics Tools* **8**, 1-15 (2003). DOI: 10.1080/10867651.2003.10487582.
- Plotly Technologies Inc. *Mesh3d* API documentation. Plotly is used only as the optional interactive triangle renderer.
- Frechet, M. “Les elements aleatoires de nature quelconque dans un espace distancee.” *Annales de l’Institut Henri Poincare* **10** (1948), 215-310.
- Karcher, H. “Riemannian center of mass and mollifier smoothing.” *Communications on Pure and Applied Mathematics* **30** (1977), 509-541. DOI: 10.1002/cpa.3160300502.

14 Euclidean metric and density resolution

Framework density uses the same Cartesian metric and fractional-grid sampling as atomic density. The scene renderer preserves equal Cartesian units, but an under-resolved Gaussian on an oblique fractional grid can distort the extracted mesh. The implementation therefore warns when the Gaussian bandwidth divided by the longest real-space grid edge is below 1.5.

Version 0.19.39a0 uses a nominal target grid interval of 0.20 angstrom and derives the Gaussian from $\sigma = 2h_{\max}$. The resolved shape scales with the display cell; the nominal Na-LTA primitive grid is 87^3 with $\sigma \approx 0.399$ angstrom. Spread-aware refinement may choose a finer grid and smaller Gaussian, subject to the voxel budget. Smooth precomputed mesh shells are the default renderer; the voxel cloud remains an optional browser fallback.